```python
import os
"""
Telegram bot для аренды парковочных мест и гаражей
С поддержкой геолокации и расчётом расстояния

Запуск: python bot.py
Требования: pip install python-telegram-bot==20.7
"""

import logging
import sqlite3
import math
from telegram import (
    Update, InlineKeyboardButton, InlineKeyboardMarkup,
    ReplyKeyboardMarkup, KeyboardButton
)
from telegram.ext import (
    Application, CommandHandler, CallbackQueryHandler,
    MessageHandler, ConversationHandler, ContextTypes, filters
)

# ── Настройки ──────────────────────────────────────────────
BOT_TOKEN = os.environ.get("BOT_TOKEN", "ВАШ_ТОКЕН_ЗДЕСЬ")
DB_FILE = "parking.db"

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ── Состояния диалога ──────────────────────────────────────
(
    MAIN_MENU,
    AD_TYPE, AD_CITY, AD_ADDRESS, AD_LOCATION, AD_PRICE, AD_DESCRIPTION, AD_PHOTO
    SEARCH_TYPE, SEARCH_LOCATION,
    CONTACT_MSG,
) = range(12)

# ── База данных ────────────────────────────────────────────
def init_db():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.executescript("""
        CREATE TABLE IF NOT EXISTS listings (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            owner_id    INTEGER NOT NULL,
```

```
            owner_name  TEXT,
            type        TEXT NOT NULL,
            city        TEXT NOT NULL,
            address     TEXT NOT NULL,
            lat         REAL,
            lon         REAL,
            price       REAL NOT NULL,
            description TEXT,
            photo_id    TEXT,
            active      INTEGER DEFAULT 1,
            created_at  TEXT DEFAULT (datetime('now'))
        );
        CREATE TABLE IF NOT EXISTS messages (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            listing_id  INTEGER NOT NULL,
            from_id     INTEGER NOT NULL,
            from_name   TEXT,
            text        TEXT,
            created_at  TEXT DEFAULT (datetime('now'))
        );
    """)
    conn.commit()
    conn.close()

def db():
    return sqlite3.connect(DB_FILE)

# ── Формула Haversine ──────────────────────────────────────────
def haversine(lat1, lon1, lat2, lon2) -> float:
    """Расстояние в метрах между двумя координатами."""
    R = 6_371_000
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlam
    return 2 * R * math.asin(math.sqrt(a))

def format_distance(meters: float) -> str:
    if meters < 1000:
        return f"{round(meters / 10) * 10:.0f} м от вас"
    elif meters < 10000:
        return f"{meters / 1000:.1f} км от вас"
    else:
        return f"{meters / 1000:.0f} км от вас"

# ── Вспомогательные функции ────────────────────────────────────
def type_label(t):
```

```
    return "🅿️ Парковочное место" if t == "parking" else "🏠 Гараж"

def listing_text(row, distance_m: float = None) -> str:
    lid, owner_id, owner_name, ltype, city, address, lat, lon, price, desc, photo
    lines = [f"{type_label(ltype)}"]
    if distance_m is not None:
        lines.append(f"📏 *{format_distance(distance_m)}*")
    lines.append(f"📍 {city}, {address}")
    lines.append(f"💰 {price:,.0f} ₽/мес")
    if desc:
        lines.append(f"📝 {desc}")
    lines.append(f"🆔 #{lid}")
    return "\n".join(lines)

def main_menu_keyboard():
    return ReplyKeyboardMarkup([
        ["📋 Подать объявление", "🔍 Найти место"],
        ["📂 Мои объявления", "ℹ️ Помощь"],
    ], resize_keyboard=True)

def geo_keyboard():
    return ReplyKeyboardMarkup([
        [KeyboardButton("📍 Отправить геолокацию объекта", request_location=True)
        [KeyboardButton("⏩ Пропустить")],
    ], resize_keyboard=True, one_time_keyboard=True)

def search_geo_keyboard():
    return ReplyKeyboardMarkup([
        [KeyboardButton("📍 Отправить мою геолокацию", request_location=True)],
        [KeyboardButton("🏙️ Искать по городу (без геолокации)")],
    ], resize_keyboard=True, one_time_keyboard=True)

# ── /start ──────────────────────────────────────────────
async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "🚗 *ParkRent Bot* — аренда парковок и гаражей\n\nВыберите действие:",
        parse_mode="Markdown",
        reply_markup=main_menu_keyboard()
    )
    return MAIN_MENU

# ── Главное меню ────────────────────────────────────────
async def main_menu(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    text = update.message.text

    if text == "📋 Подать объявление":
        keyboard = InlineKeyboardMarkup([
```

```
            [InlineKeyboardButton("🅿️ Парковочное место", callback_data="ad_type_
            [InlineKeyboardButton("🏠 Гараж", callback_data="ad_type_garage")],
        ])
        await update.message.reply_text("Что вы сдаёте?", reply_markup=keyboard)
        return AD_TYPE

    elif text == "🔍 Найти место":
        keyboard = InlineKeyboardMarkup([
            [InlineKeyboardButton("🅿️ Парковочное место", callback_data="search_p
            [InlineKeyboardButton("🏠 Гараж", callback_data="search_garage")],
            [InlineKeyboardButton("📋 Всё", callback_data="search_all")],
        ])
        await update.message.reply_text("Что ищете?", reply_markup=keyboard)
        return SEARCH_TYPE

    elif text == "📁 Мои объявления":
        return await show_my_listings(update, ctx)

    elif text == "ℹ️ Помощь":
        await update.message.reply_text(
            "*Как пользоваться ботом:*\n\n"
            "📋 *Подать объявление* — укажите адрес и точку на карте\n"
            "🔍 *Найти место* — бот покажет расстояние от вас до каждого объекта\
            "📁 *Мои объявления* — управление вашими объявлениями\n\n"
            "💡 При поиске отправьте геолокацию — бот отсортирует по близости к в
            parse_mode="Markdown",
            reply_markup=main_menu_keyboard()
        )
        return MAIN_MENU

    return MAIN_MENU

# ── Подача объявления ──────────────────────────────────────────
async def ad_type_chosen(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    ctx.user_data["ad"] = {"type": query.data.replace("ad_type_", "")}
    await query.edit_message_text("🏙️ Введите *город*:", parse_mode="Markdown")
    return AD_CITY

async def ad_city(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data["ad"]["city"] = update.message.text.strip()
    await update.message.reply_text("📍 Введите *адрес* (улица, дом, ориентир):",
    return AD_ADDRESS

async def ad_address(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data["ad"]["address"] = update.message.text.strip()
```

```
    await update.message.reply_text(
        "🗺️ Теперь укажите *точное местоположение* объекта на карте.\n\n"
        "Нажмите кнопку ниже или «Пропустить» — тогда расчёт расстояния будет нед
        parse_mode="Markdown",
        reply_markup=geo_keyboard()
    )
    return AD_LOCATION

async def ad_location_received(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    loc = update.message.location
    ctx.user_data["ad"]["lat"] = loc.latitude
    ctx.user_data["ad"]["lon"] = loc.longitude
    await update.message.reply_text(
        f"✅ Местоположение сохранено!\n🗺️ {loc.latitude:.5f}, {loc.longitude:.5f
        "💰 Введите *цену в месяц* (только число, ₽):",
        parse_mode="Markdown",
        reply_markup=main_menu_keyboard()
    )
    return AD_PRICE

async def ad_location_skipped(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data["ad"]["lat"] = None
    ctx.user_data["ad"]["lon"] = None
    await update.message.reply_text(
        "⏩ Геолокация пропущена.\n\n💰 Введите *цену в месяц* (только число, ₽):
        parse_mode="Markdown",
        reply_markup=main_menu_keyboard()
    )
    return AD_PRICE

async def ad_price(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    try:
        price = float(update.message.text.strip().replace(" ", "").replace(",", "
        ctx.user_data["ad"]["price"] = price
    except ValueError:
        await update.message.reply_text("❌ Введите число, например: 5000")
        return AD_PRICE
    await update.message.reply_text(
        "📝 Добавьте *описание* (особенности, размер, доступ)\n"
        "Или «-» чтобы пропустить:", parse_mode="Markdown"
    )
    return AD_DESCRIPTION

async def ad_description(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip()
    ctx.user_data["ad"]["description"] = None if text == "-" else text
    await update.message.reply_text(
```

```
        "📸 Отправьте *фото* (необязательно)\nИли «-» чтобы пропустить:",
        parse_mode="Markdown"
    )
    return AD_PHOTO

async def ad_photo(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if update.message.photo:
        ctx.user_data["ad"]["photo_id"] = update.message.photo[-1].file_id
    else:
        ctx.user_data["ad"]["photo_id"] = None

    ad = ctx.user_data["ad"]
    has_geo = ad.get("lat") is not None
    geo_line = f"🗺️ {ad['lat']:.5f}, {ad['lon']:.5f}" if has_geo else "🗺️ Геолока

    preview = (
        f"*Проверьте объявление:*\n\n"
        f"{type_label(ad['type'])}\n"
        f"📍 {ad['city']}, {ad['address']}\n"
        f"{geo_line}\n"
        f"💰 {ad['price']:,.0f} ₽/мес\n"
    )
    if ad.get("description"):
        preview += f"📝 {ad['description']}\n"
    if ad.get("photo_id"):
        preview += "🖼️ Фото прикреплено\n"

    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton("✅ Опубликовать", callback_data="ad_publish")],
        [InlineKeyboardButton("❌ Отмена", callback_data="ad_cancel")],
    ])
    await update.message.reply_text(preview, parse_mode="Markdown", reply_markup=
    return AD_CONFIRM

async def ad_publish(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    if query.data == "ad_cancel":
        await query.edit_message_text("❌ Объявление отменено.")
        return MAIN_MENU

    ad = ctx.user_data.get("ad", {})
    user = query.from_user
    conn = db()
    conn.execute(
        "INSERT INTO listings (owner_id, owner_name, type, city, address, lat, lo
```

```
        "VALUES (?,?,?,?,?,?,?,?,?,?)",
        (user.id, user.full_name, ad["type"], ad["city"], ad["address"],
         ad.get("lat"), ad.get("lon"), ad["price"], ad.get("description"), ad.get
    )
    conn.commit()
    conn.close()
    await query.edit_message_text("✅ Объявление опубликовано!")
    return MAIN_MENU

# ── Поиск ──────────────────────────────────────────────────────
async def search_type_chosen(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    ctx.user_data["search_type"] = query.data.replace("search_", "")
    await query.edit_message_text(
        "📍 Отправьте вашу геолокацию — бот покажет расстояние до каждого объекта
        "Или выберите поиск по названию города.",
    )
    await query.message.reply_text("Выберите способ поиска:", reply_markup=search
    return SEARCH_LOCATION

async def search_with_location(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    """Пользователь отправил геолокацию."""
    loc = update.message.location
    user_lat, user_lon = loc.latitude, loc.longitude
    ltype = ctx.user_data.get("search_type", "all")

    conn = db()
    if ltype == "all":
        rows = conn.execute("SELECT * FROM listings WHERE active=1").fetchall()
    else:
        rows = conn.execute("SELECT * FROM listings WHERE active=1 AND type=?", (
    conn.close()

    if not rows:
        await update.message.reply_text("😕 Объявлений пока нет.", reply_markup=m
        return MAIN_MENU

    # Разделяем на «с геолокацией» и «без»
    with_geo = []
    without_geo = []
    for row in rows:
        lat, lon = row[6], row[7]
        if lat is not None and lon is not None:
            dist = haversine(user_lat, user_lon, lat, lon)
            with_geo.append((dist, row))
        else:
```

```
            without_geo.append(row)

    with_geo.sort(key=lambda x: x[0])  # сортировка по близости
    total = len(with_geo) + len(without_geo)

    await update.message.reply_text(
        f"🔍 Найдено {total} объявл. — отсортированы по расстоянию от вас 📏",
        reply_markup=main_menu_keyboard()
    )

    for dist, row in with_geo:
        lid, owner_id = row[0], row[1]
        caption = listing_text(row, distance_m=dist)
        buttons = [
            [InlineKeyboardButton("🗺️ Открыть на карте", callback_data=f"map_{lid
            [InlineKeyboardButton("✉️ Написать арендодателю", callback_data=f"con
        ]
        keyboard = InlineKeyboardMarkup(buttons)
        if row[10]:
            await update.message.reply_photo(row[10], caption=caption, parse_mode
        else:
            await update.message.reply_text(caption, parse_mode="Markdown", reply

    if without_geo:
        await update.message.reply_text("📌 *Объявления без геолокации:*", parse_
        for row in without_geo:
            lid, owner_id = row[0], row[1]
            caption = listing_text(row)
            keyboard = InlineKeyboardMarkup([
                [InlineKeyboardButton("✉️ Написать арендодателю", callback_data=f
            ])
            if row[10]:
                await update.message.reply_photo(row[10], caption=caption, parse_
            else:
                await update.message.reply_text(caption, parse_mode="Markdown", r

    return MAIN_MENU

async def search_by_city_prompt(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "🏙️ Введите *город* для поиска:", parse_mode="Markdown",
        reply_markup=main_menu_keyboard()
    )
    return SEARCH_LOCATION

async def search_city_input(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    city = update.message.text.strip()
```

```
    ltype = ctx.user_data.get("search_type", "all")

    conn = db()
    if ltype == "all":
        rows = conn.execute(
            "SELECT * FROM listings WHERE active=1 AND lower(city)=lower(?) ORDER
            (city,)
        ).fetchall()
    else:
        rows = conn.execute(
            "SELECT * FROM listings WHERE active=1 AND lower(city)=lower(?) AND t
            (city, ltype)
        ).fetchall()
    conn.close()

    if not rows:
        await update.message.reply_text(
            f"😕 Объявлений в городе *{city}* не найдено.",
            parse_mode="Markdown", reply_markup=main_menu_keyboard()
        )
        return MAIN_MENU

    await update.message.reply_text(
        f"🔍 Найдено {len(rows)} объявл. в *{city}*\n"
        f"💡 _Для расчёта расстояний — отправьте геолокацию при поиске_",
        parse_mode="Markdown"
    )

    for row in rows:
        lid, owner_id = row[0], row[1]
        caption = listing_text(row)
        buttons = [[InlineKeyboardButton("✉️ Написать арендодателю", callback_dat
        if row[6] is not None:
            buttons.insert(0, [InlineKeyboardButton("🗺️ Открыть на карте", callba
        keyboard = InlineKeyboardMarkup(buttons)
        if row[10]:
            await update.message.reply_photo(row[10], caption=caption, parse_mode
        else:
            await update.message.reply_text(caption, parse_mode="Markdown", reply

    return MAIN_MENU

# ── Показать на карте ──────────────────────────────────────────
async def show_map(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    lid = int(query.data.split("_")[1])
```

```
    conn = db()
    row = conn.execute("SELECT lat, lon, address FROM listings WHERE id=?", (lid,
    conn.close()
    if row and row[0]:
        await query.message.reply_location(latitude=row[0], longitude=row[1])
        await query.message.reply_text(f"📍 {row[2]}")
    else:
        await query.answer("Геолокация не указана в объявлении", show_alert=True)
    return MAIN_MENU

# ── Связь с арендодателем ──────────────────────────────────────
async def contact_start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    parts = query.data.split("_")
    lid, owner_id = int(parts[1]), int(parts[2])
    ctx.user_data["contact_listing"] = lid
    ctx.user_data["contact_owner"] = owner_id

    if query.from_user.id == owner_id:
        await query.answer("Это ваше объявление!", show_alert=True)
        return MAIN_MENU

    await query.message.reply_text(
        "✉️ Напишите сообщение арендодателю:",
        reply_markup=main_menu_keyboard()
    )
    return CONTACT_MSG

async def contact_send(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    lid = ctx.user_data.get("contact_listing")
    owner_id = ctx.user_data.get("contact_owner")
    user = update.effective_user
    text = update.message.text

    conn = db()
    conn.execute(
        "INSERT INTO messages (listing_id, from_id, from_name, text) VALUES (?,?,
        (lid, user.id, user.full_name, text)
    )
    conn.commit()
    conn.close()

    try:
        username_info = f"@{user.username}" if user.username else f"ID: {user.id}
        await ctx.bot.send_message(
            owner_id,
```

```
            f"📩 *Новое сообщение* по объявлению #{lid}\n"
            f"От: {user.full_name} ({username_info})\n\n{text}",
            parse_mode="Markdown"
        )
        await update.message.reply_text("✅ Сообщение отправлено!", reply_markup=
    except Exception:
        await update.message.reply_text(
            "✅ Сообщение сохранено. Арендодатель получит его при следующем входе
            reply_markup=main_menu_keyboard()
        )
    return MAIN_MENU

# ── Мои объявления ──────────────────────────────────────────
async def show_my_listings(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    conn = db()
    rows = conn.execute(
        "SELECT * FROM listings WHERE owner_id=? ORDER BY id DESC", (user_id,)
    ).fetchall()
    conn.close()

    if not rows:
        await update.message.reply_text(
            "У вас нет объявлений.\nНажмите «📋 Подать объявление»",
            reply_markup=main_menu_keyboard()
        )
        return MAIN_MENU

    await update.message.reply_text(f"📋 Ваши объявления ({len(rows)}):")
    for row in rows:
        lid = row[0]
        active = row[11]
        has_geo = row[6] is not None
        status = "✅ Активно" if active else "⏸ Снято"
        geo_status = "🗺 с геолокацией" if has_geo else "📌 без геолокации"
        caption = listing_text(row) + f"\n{status} · {geo_status}"

        buttons = []
        if active:
            buttons.append(InlineKeyboardButton("⏸ Снять", callback_data=f"deact
        else:
            buttons.append(InlineKeyboardButton("▶ Активировать", callback_data=
        buttons.append(InlineKeyboardButton("🗑 Удалить", callback_data=f"delete_
        keyboard = InlineKeyboardMarkup([buttons])

        if row[10]:
            await update.message.reply_photo(row[10], caption=caption, parse_mode
```

```
        else:
            await update.message.reply_text(caption, parse_mode="Markdown", reply

    return MAIN_MENU

async def manage_listing(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    parts = query.data.split("_")
    action, lid = parts[0], int(parts[1])
    user_id = query.from_user.id

    conn = db()
    row = conn.execute("SELECT owner_id FROM listings WHERE id=?", (lid,)).fetcho
    if not row or row[0] != user_id:
        await query.answer("Нет доступа", show_alert=True)
        conn.close()
        return MAIN_MENU

    if action == "deactivate":
        conn.execute("UPDATE listings SET active=0 WHERE id=?", (lid,))
        conn.commit()
        await query.answer("⏸️ Объявление снято с публикации")
    elif action == "activate":
        conn.execute("UPDATE listings SET active=1 WHERE id=?", (lid,))
        conn.commit()
        await query.answer("✅ Объявление активно")
    elif action == "delete":
        conn.execute("DELETE FROM listings WHERE id=?", (lid,))
        conn.commit()
        await query.answer("🗑️ Удалено")
        await query.message.reply_text(f"🗑️ Объявление #{lid} удалено.")

    conn.close()
    await query.edit_message_reply_markup(None)
    return MAIN_MENU

# ── Запуск ──────────────────────────────────────────────────
def main():
    init_db()
    app = Application.builder().token(BOT_TOKEN).build()

    conv = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            MAIN_MENU: [
                MessageHandler(filters.TEXT & ~filters.COMMAND, main_menu),
```

```
                CallbackQueryHandler(manage_listing, pattern="^(deactivate|activa
                CallbackQueryHandler(contact_start, pattern="^contact_"),
                CallbackQueryHandler(show_map, pattern="^map_"),
            ],
            AD_TYPE: [CallbackQueryHandler(ad_type_chosen, pattern="^ad_type_")],
            AD_CITY: [MessageHandler(filters.TEXT & ~filters.COMMAND, ad_city)],
            AD_ADDRESS: [MessageHandler(filters.TEXT & ~filters.COMMAND, ad_addre
            AD_LOCATION: [
                MessageHandler(filters.LOCATION, ad_location_received),
                MessageHandler(filters.Regex("^⏩"), ad_location_skipped),
            ],
            AD_PRICE: [MessageHandler(filters.TEXT & ~filters.COMMAND, ad_price)]
            AD_DESCRIPTION: [MessageHandler(filters.TEXT & ~filters.COMMAND, ad_d
            AD_PHOTO: [
                MessageHandler(filters.PHOTO, ad_photo),
                MessageHandler(filters.TEXT & ~filters.COMMAND, ad_photo),
            ],
            AD_CONFIRM: [CallbackQueryHandler(ad_publish, pattern="^ad_(publish|c
            SEARCH_TYPE: [CallbackQueryHandler(search_type_chosen, pattern="^sear
            SEARCH_LOCATION: [
                MessageHandler(filters.LOCATION, search_with_location),
                MessageHandler(filters.Regex("^🌆"), search_by_city_prompt),
                MessageHandler(filters.TEXT & ~filters.COMMAND, search_city_input
            ],
            CONTACT_MSG: [MessageHandler(filters.TEXT & ~filters.COMMAND, contact
        },
        fallbacks=[CommandHandler("start", start)],
        allow_reentry=True,
    )

    app.add_handler(conv)
    print("🤖 Бот запущен! Нажмите Ctrl+C для остановки.")
    app.run_polling()

if __name__ == "__main__":
    main()
```